

CS4243 Cheatsheet

Emilia Ayu

Variables and Operations

Statistics

Mean / Std. dev.

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i \quad \sigma = \sqrt{\frac{1}{n} \sum_i (x_i - \bar{x})^2}$$

Normalisation $x' = \frac{x - \mu}{\sigma}$ (or min-max: $\frac{x - \min}{\max - \min}$)

Gaussian (1D) $G(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-x^2/2\sigma^2}$

Gaussian (2D) $G(x, y) = \frac{1}{2\pi\sigma^2} e^{-(x^2+y^2)/2\sigma^2}$

Accuracy: $\frac{TP + TN}{TP + TN + FP + FN}$ **Precision:** $\frac{TP}{TP + FP}$

Recall: $\frac{TP}{TP + FN}$ **F1-score:** $\frac{2PR}{P + R}$

Calculus

Line eqns: $y = mx + b$; $x \cos \theta + y \sin \theta = \rho$ **Euclidean dist:** $d = \sqrt{\sum_i (x_i - y_i)^2}$

Gradient: $\nabla f = \left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right)$ **Laplacian:** $\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$

Orientation: $\theta = \tan^{-1} \left(\frac{f_y}{f_x} \right)$ **Magnitude:** $\|\nabla f\| = \sqrt{f_x^2 + f_y^2}$

Lecture 1 - Computer Vision

Digital Image Acquisition

Illuminance $(i(x, y), 0 \leq i \leq \infty)$: amount of source illumination incident on scene.

Reflectance $(r(x, y), 0 \leq r \leq 1)$: amount of source illumination reflected by objects on scene.

Exposure: amount of time incident light can reach imaging sensor.

Aperture: how much light you let in (too much: bokeh, too narrow sharp image)

Shutter Speed: how much time you give light to accumulate (slow: motion blur, fast: freeze action)

	Spatial Resolution	Intensity Resolution
Definition	smallest discernible detail	smallest discernible δ intensity level
Measurement	pixels / unit distance	bit depth / pixel value
Determined by	sampling	quantisation
Effects of low res	pixellation	false contouring

File size (bits): $b = M \cdot N \cdot k$

M : rows, N : columns, k : bits/pixel, b : total bits

File Extensions

.bmp: raw bit file (largest size)

.png: predicts pixels based on neighbours, reconstructs original pixel value (lossless)

.jpg: 8x8 image blocks represented by 64 Discrete Cosine Transform coefficients, reconstruction is approximate (lossy). Zigzag ordering captures larger coefficients first; more efficient for Huffman entropy encoding

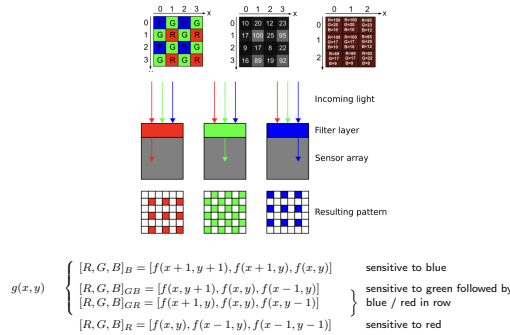
Colour Representation

RGB Space $im(x, y, b) = y$ pixels down, x pixels to right in the b th channel

HSV Space Hue Saturation Value

Bayer Filter

Each pixel only stores one color sample over it (RGB). Reduces intensity resolution.



Lecture 2 - Point Processing and Filtering

Point processing: $x_{ij} = f(p_{ij})$

Filtering: $x_{ij} = f(w_{ij})$

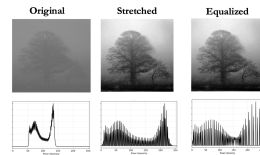
Point Processing

Brightness = $x_{ij} = p_{ij} + b$ (clip when needed)

Intensity scaling = $x_{ij} = a \cdot p_{ij}$

Image Standardisation Preprocess to remove arbitrary brightness/contrast baselines. Algorithms process better when inputs are centered around 0.

Histogram Stretching vs. Equalisation



Stretching: linear rescale to full range

$$s = \frac{r - r_{\min}}{r_{\max} - r_{\min}} (L - 1)$$

Pros: improves contrast; preserves relative brightness relationships

Cons: limited effect if histogram already spans most of the range

Equalisation: redistribute via CDF for uniform histogram

$$s = (L - 1) \sum_{k=0}^r p(k), \quad s_{\text{output}} = k \cdot CDF(s_{\text{input}})$$

Pros: more dramatic contrast change; reveals poorly represented ranges automatically

Cons: can amplify noise; alters clinically irrelevant contrast

Cross-Correlation and Convolution

Purpose: smoothing/averaging with neighbours reduces impulse (salt-and-pepper) noise; also used for edge detection and template matching.

Cross-Correlation: slide kernel F as-is, multiply & sum

$$x_{ij} = \sum_u \sum_v f_{uv} \cdot p_{i+u, j+v}, \quad X = P \circledast F$$

Convolution: flip kernel 180° first, then multiply & sum. Commutative and associative.

$$x_{ij} = \sum_u \sum_v f_{uv} \cdot p_{i-u, j-v}, \quad X = P * F$$

Equal when: kernel is symmetric under 180° rotation, i.e. $F = F_{\text{flip}}$ (e.g. box filter, Gaussian).

Gaussian Kernel

Used as the filter f_{uv} in cross-correlation/convolution above to smooth/blur. Larger σ = more blur.

Detail extraction (unsharp masking):

Details = Original - Blurred $\Rightarrow$ Sharpened = Original + α · Details

Lecture 3 - Gradients

Sobel Filter

Horizontal Sobel filter: $\begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} * \begin{bmatrix} 1 & 0 & -1 \\ 0 & 0 & 0 \\ 1 & 0 & -1 \end{bmatrix}$ 1D derivative filter. $\nabla f = \left[\frac{\partial f}{\partial x}, 0 \right]$

Vertical Sobel filter: $\begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix} = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix} * \begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix}$ 1D derivative filter. $\nabla f = \left[0, \frac{\partial f}{\partial y} \right]$

Vertical filter S_y detects horizontal edges; horizontal filter S_x detects vertical edges (both across gradient). The gradient points across the edge, ie direction is normal to the edge. Gradient will point from darker to brighter intensities. The edge occurs at peaks. A 2D filter is **separable** if it can be written as an outer product of a column vector and a row vector:

$$K = u v^T$$

Convolving with the full 2D kernel K is equivalent to convolving with u then v separately. Cheaper since $1 \times n + n \times 1$ multiplications beat $n \times n$.

Example — Vertical Sobel filter:

- Smooth horizontally: $[1, 2, 1]$
- Differentiate vertically: $[1, 0, -1]^T$

$$S_y = \begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix} \begin{bmatrix} 1 & 2 & 1 \end{bmatrix}$$

Computing gradients

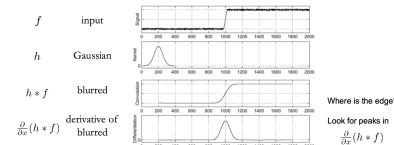
- Convolve gradient filter with image f to compute image gradient components

$$G_x = \frac{\partial f}{\partial x} = S_x * f \quad G_y = \frac{\partial f}{\partial y} = S_y * f$$

- Compute gradient orientation and magnitude

Gaussian Derivative and Laplace Filters

Differentiation accentuates noise (multiplicative factor proportional to frequency) $\Rightarrow$ blur before using derivative features to remove noise (but the edge will blur).



Stronger Gaussian returns more high-frequency noise $\rightarrow$ spreads edge across larger region.

Differentiation Property: $\frac{\partial}{\partial x} (h * f) = \left(\frac{\partial h}{\partial x} \right) * f$

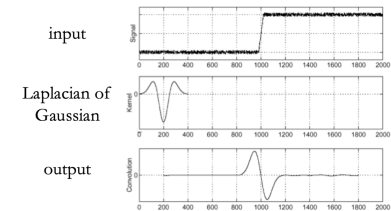
Differentiation is a convolution operation: $f * [1 \ 0 \ -1]$ (x-direction) or $f * [1 \ 0 \ -1]^T$ (y-direction)

Laplacian Operator

Continuous: $\Delta f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$

Discrete: $\nabla^2 f(x, y) = f(x+1, y) + f(x-1, y) + f(x, y+1) + f(x, y-1) - 4f(x, y)$

Laplacian of Gaussian Filter



Zero crossings. Localise edges more accurately but less convenient to use.

Canny Edge Detector

Deriv. of Gaussian $\rightarrow$ Grad. mag/orient. $\rightarrow$ NMS $\rightarrow$ Hysteresis thresh.

Non-Maximum Suppression: Only keep local maxima for edges (so edges will be one pixel wide). *Algorithm* Given magnitude map $M(x, y)$ and orientation map $\theta(x, y)$, for every (x, y)

- Find two neighbours in that direction according to the nearest angle $(+\theta, -\theta)$
- Interpolate the two neighbour values, with value $= x * (a - b) + b$, where a, b are the two real neighbour pixel magnitudes, and x is the fractional distance, $x = \tan(\theta \text{ mod } 90)$



- Compare centre pixel to interpolated neighbours.

If $M(x, y) \geq$ both neighbours: keep $M(x, y)$ Else: suppress and set $M(x, y) = 0$

Hysteresis Thresholding: Set two thresholds, H to start edge curves, and L to grow and link edges (if connected to boundary, mark said pixel as boundary)

Algorithm E_H : set of all values passing H , E_L : set of all values passing L . Let $D = E_L - E_H$ (keep only edges of D connected to E_H)

Final image: $E_H \cup D'$

Hough Transform Algorithm Given magnitude map $M(x, y)$ and orientation map $\theta(x, y)$, for every (x, y)

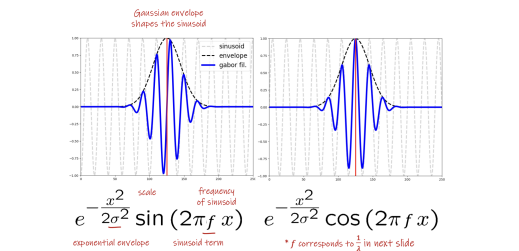
- Quantise parameter space (m, b) by picking a range and step size, then discretise.
- Initialise accumulator array $A(m, b)$. Set each $A(m, b) = 0, \forall m, b$.
- $\forall (x_i, y_i), \forall m : b : b = x_i m + y_i$. Increment $A(m, b) = A(m, b) + 1$
- Threshold and find local maxima of $A(m, b)$. $y = m * x + b^*$

Use $x \cos \theta + y \sin \theta = \rho$ — finite parameters ($\theta \in [0, 360]$) vs. unbounded gradient m

Can be made more efficient by voting only near its measured gradient orientation.

Lecture 4 - Regions

Filter banks: collection of multiple filter kernels, where one filter kernel responds to one local pattern. **Gabor Filter:** fix general math function with a range of parameters. Composed of sinuisoids and Gaussian envelope.



f: frequency selects spacing between waves; sigma: scale determines size of region covered by Gaussian envelope. **2D Gabor formula:**

fmn = (1 / (2*pi*sigma^2)) * exp[-(m^2 + gamma^2*n^2) / (2*sigma^2)] * sin[(2*pi*cos[omega]*m + sin[omega]*n) / lambda + phi]

Labels: exponential (Gaussian) envelope selects a local area; 2D-Sinusoid selects orientation and frequency

omega - orientation of sinusoid in 2D
lambda - wavelength of sinusoid (inversely proportional to f)
phi - phase of sinusoid; shift from center of kernel
gamma - aspect ratio of envelope coverage (gamma = 1 means circular, otherwise elliptical)
sigma - rate of decay of exponential envelope

Multi-Scale Reasoning

Smoothing suppresses details, which we want to ignore variations in smaller distances to focus on larger structure. *Gaussian image pyramid* -> progressively blur then downsample at fixed stride or downsampling rate. We should smooth before downsampling to prevent aliasing (high-frequency can mask as low frequency, ringing).

Scale selection Measure responses on multiple scales. Choose maximum filter size that matches structure size to detect structures.

Single vs Multi-scale Representation Given ri, rhoi corresponding to response and scale at i.
Single scale: r = [r1, r2, ..., rk]
Multi-scale: r = [r1^rho1, r2^rho1, ..., rk^rho1, r1^rho2, ..., rk^rho2].

Textures

Textures occur over an area. To detect, group pixels by region Omega and average filter responses over region.

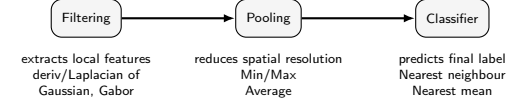
- 1. r(x, y) = [r1(x, y), r2(x, y), ..., rk(x, y)] Run k different filters to get k responses.
- 2. tk(Omega) = (1/|Omega|) * sum |rk(x, y)|. Take absolute value of every pixel's response to filter k in region Omega and sum. Divide pixels by |Omega| for average.
tk(Omega) is a pooled response of filter k across Omega

3. t(Omega) = [t1(Omega), t2(Omega), ..., tk(Omega)]^T

To compare textures, D(OmegaA, OmegaB) = ||t(OmegaA) - t(OmegaB)||, i.e. D is the Euclidean distance.

Classifiers

- 1. Nearest Neighbour Classifier: i* = arg min_i ||t(Omega) - t(Omega_i)|| where c-hat = yi*
- 2. Nearest Class Mean: mu_c = (1/N_i) * sum i : yi = ct(Omega_c) where c-hat(Omega) = arg min ||t(Omega) - mu_c||

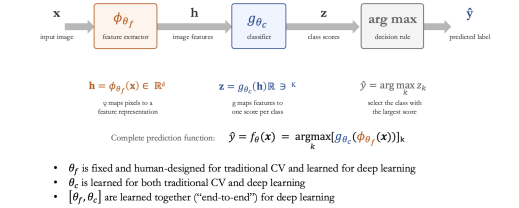


Topic 5 - CNNs

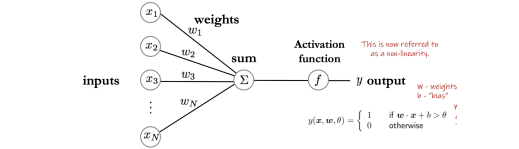
Receptive field: set of image pixels an intermediate output pixel depends on.

Recursive formula (per layer l, with kernel kl, stride sl):
rl = rl-1 + (kl - 1)jl-1, jl = jl-1 sl
base case: r0 = 1, j0 = 1

Training: Training Images -> Image Features -> Training with Labels -> Learned Classifier
Testing: Test Image -> Image Features -> Learned Classifier -> Prediction
Stages of Classifier



Perceptron



No of learnable parameters = weights (edges) + perceptrons (neurons).
Neurons are total of all possible output nodes (all layers except input). We add perceptrons as bias terms.

MLP Formalism

Input: a^(0) = u; u: input feature vector
Hidden layers:

a^(l) = sigma(W^(l)a^(l-1) + b^(l)), l = 1, ..., L - 1
a^(l): representation in layer l.
W^(l): weights in layer l.
b^(l): biases in layer l.
sigma: non-linear activation function
Output layer: z = a^(L) = W^(L)a^(L-1) + b^(L)

Convolution

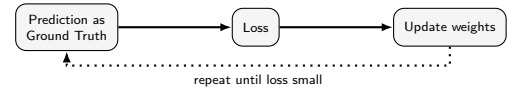
No of learnable parameters:
k^2 * Cin * Cout + Cout

Strided. Downsample each feature map. Reduce spatial resolution. **Dilated.** Enlarge receptive field without adding params or reducing resolution.

H_output = floor((H_input + 2P - D(K - 1) - 1) / S) + 1

Logits: unrestricted real-value scales -> convert into probability distribution using softmax.

Loss functions



Examples

- 1. Localisation loss: L_loc = (x-hat - x)^2 + (y-hat - y)^2. Distance-based, naturally continuous.
- 2. L0/1(y-hat, y) = { 0, y-hat = y (correct); 1, y-hat != y (wrong) }. Discrete
- 3. Cross-entropy loss: L_CE = -log(py). Continuous -> softly penalizes low confidence in the correct class

Object Detection

Window-based: Build/train model -> Generate candidate windows -> Score candidates -> Resolve detections with NMS.

Performance Evaluation Area of Overlap Measure
AO(Bgt, Bp) = |Bgt intersect Bp| / |Bgt union Bp|

mAP (mean Average Precision): standard object-detection metric, 0-1 or 0-100%. We want high precision (few false positives) and high recall (few false negatives).

- AP(c, tau) = area under precision-recall curve (class c, IoU >= tau)
- mAP(tau) = mean AP across classes

AP(c, tau) = integral from 0 to 1 of p(r) dr mAP(tau) = (1/|C|) * sum over c in C of AP(c, tau)

CNN-Based Object Detection

Topic 6 - Segmentation

Separation of images into regions/objects.
Semantic: each pixel has a class label. Identify foreground and background.
Instance: Each pixel has an object it belongs to.
Panoptic: Combine semantic and instance.
Pixel affinity: evidence that makes two pixels belong to the same region (color, texture, etc.).
k-Means Clustering: Iteratively re-assign points to nearest cluster center
1. Randomly initialize the cluster centers c1, ..., cK
2. For each point pi, find the closest cj to put pi in

- 3. Set cj to be mean of points in cluster j
- 4. Repeat, if cj have changed up to some threshold
 - Pros: Simple, Converges to local min.
 - Cons: Setting K, Sensitive to initial centers (Since k-means converges to local min.), Sensitive to outliers (Can add more clusters), Assumes spherical clusters

Simple Linear Iterative Clustering (SLIC)

- **Superpixel:** Group of pixels that share common traits. Compact and meaningful representation.
- No of pixels: ntp; Target no of superpixels: nsp
- Initial width of each superpixel: s = sqrt(ntp/nsp)
- Features: z = [r, g, b, x, y]
- Color distance: dc = ||(rj, gj, bj) - (ri, gi, bi)||
- Spatial distance: ds = ||(xj, yj) - (xi, yi)||
- dcm and dsm = s set as max. expected values of dc and ds respectively
- D = sqrt((dc/dcm)^2 + (ds/dsm)^2) = sqrt(d_c^2 + (ds/s)^2 * c^2)

- 1. Split img. into grid of size s x s. Set cluster centers as lowest gradient position in 3 x 3 neighborhood from superpixel center.
- 2. For each cluster center, check distance to all pixels within 2s x 2s neighborhood. Assign pixels to closest checked center.
- 3. Update cluster centers using mean; repeat until converged (Same as k-Means)
- 4. Optional: Replace superpixel region by average value to create stained glass effect

- Modification of k-Means: Not random initialization, Compute pixel's distance only to closest set of centers

Mean-Shift Clustering: Find local density maxima in feature space

- **Attraction basin:** Region in feature space for which all trajectories of centroids lead to same mode
- **Cluster:** All data points in attraction basin of a mode
- 1. For each data point:

- (a) Define window around and get centroid
- (b) Shift window to centroid
- (c) Repeat until window centroid stops moving
- Segmentation with Mean Shift: Do mean shift and merge pixels in same attraction basin
- Choosing window size: Trial and error, Sample points and use avg. dist. to knn. (Num. of neighbors needs to be large enough to ensure increase in density) - Larger window size -> Fewer clusters

- Pros: No assumptions on cluster shape, 1 parameter, Finds variable num. of modes (vs. specified k in k-Means), Robust to outliers
- Cons: Choosing h, Slow, Scales poorly with feature space dimension
- Optimizations:
 - After each run of mean shift, assign all points within radius r of end point to same cluster
 - Assign points in radius c < r of search path to mode